VISION
VIGILANTS

IBVAP

Intelligent Border Video Analytics Platform
Technical reference for the SIH 2026 presentation

Field	Value
Team	Vision Vigilants
Problem area	AI-based intelligent border video surveillance
Codebase	20,044 lines of Python across 76 modules, plus a 2,447-line React dashboard
Automated tests	269, all passing
Language / runtime	Python 3.11 (desktop app and pipeline), Node + Vite (dashboard)
Document generated	2026-09-09

How to use this document. Everything listed under “Implemented” exists in the repository and is covered by tests. Sections marked **NOT IMPLEMENTED** or **LIMITATION** must not be presented as working features. The measured numbers are from runs on a Linux container with CPU inference; timings on an Apple Silicon Mac will differ, so quote the ratios rather than the absolute milliseconds.

1. What the system does

IBVAP takes a video feed from any camera - a file, a webcam or an RTSP stream - and turns it into security events an operator can act on. It detects and tracks people and vehicles, estimates body pose, classifies what people are doing over time, tests positions against optional per-camera geometry, scores the result for threat, and records incidents with photographic evidence in a local database.

The central architectural rule is that **AI capability is camera-agnostic and security geometry is camera-specific**. Every camera runs the identical pipeline; only the polygons differ, because a polygon describes a physical region of one camera's view. No code branches on a camera ID anywhere in the project.

Processing chain

Stage	What it produces	Module
Video input	Frames, with media-time timestamps	inputs/video_source.py, webcam_source.py, rtsp_source.py
Object detection	Boxes, classes, confidences	core/detector.py
Multi-object tracking	Persistent Track IDs	core/detector.py + core/tracker.py
Track state	Trajectory, speed, heading, dwell	core/tracker.py

Stage	What it produces	Module
Pose estimation	17 body keypoints per person	core/pose.py
Pose history	Per-Track-ID temporal buffer	core/pose_history.py
Movement analysis	Scale-free movement signals	core/movement.py
Behaviour classification	Named actions with confidence	core/behaviour_classifier.py
Scene context	Low-light state from luminance	core/low_light.py
Zone / line geometry	Intrusion and crossing events	core/zone_engine.py, core/line_crossing.py
Behaviour engine	Loitering, 8-feature vector	core/behaviour_engine.py
Anomaly detection	Unsupervised outlier score	core/anomaly_engine.py
Threat scoring	Explainable additive score	core/threat_engine.py
Incidents	Deduplicated, escalating records	core/incident_manager.py
Evidence	Annotated frames and crops on disk	core/evidence.py
Alerts	Operator notification	core/alert_manager.py
Storage	SQLite, 13 tables, 18 indexes	database/

2. AI, machine learning and deep learning

2.1 Deep learning models

Model	Type	Role	Notes
YOLOv8n	One-stage CNN object detector	Detects person, car, motorcycle, bus, truck, bicycle	Ultralytics pretrained COCO weights, 6.5 MB. Class names are read from the model at runtime, never hard-coded.
YOLOv8n-pose	CNN pose estimator, COCO-17	17 body keypoints per confirmed person	6.8 MB. Runs on the person CROP, not the whole frame, so it reuses the tracker's identity instead of finding its own people.
border_yolo.pt	Custom 7-class aerial detector	Present but not the default	Trained on DOTA + xView + VisDrone. Measured person recall 0.28 on ground-level video, so the stock model is used instead.

2.2 Classical machine learning

Algorithm	Library	Purpose
Isolation Forest	scikit-learn 1.8	Unsupervised behavioural anomaly detection over completed tracks. 150 estimators, contamination 0.10, fixed random_state 42.
StandardScaler	scikit-learn	Feature normalisation before the forest is fitted.

The anomaly model learns from **8 features per completed track**: avg_speed, max_speed, dwell_seconds, distance_norm, direction_changes, zone_transitions, restricted_seconds, track_age_seconds. It is unsupervised - nothing is labelled - and it reports `INSUFFICIENT_DATA` until it has seen 25 completed tracks rather than guessing. The fitted model is persisted with joblib and warm-started from SQLite.

2.3 Tracking

Algorithm	Purpose
ByteTrack	Multi-object tracking. Assigns a persistent integer Track ID to each detection across frames, including low-confidence detections that a naive tracker drops. Supplied by Ultralytics; BoT-SORT is also selectable.
Track confirmation (min_hits)	A track must be seen 3 times before it can raise an incident or be shown to the operator, which filters one-frame false positives.

2.4 Rule-based behaviour classification

> **State this accurately on the slide.** There is **no trained action-recognition model** in this project. Behaviour is classified by explainable geometric rules over pose history. That is a deliberate choice: every classification can be traced to the numbers that produced it, which a black box could not be. Do not describe it as a deep-learning action classifier.

Eight behaviours are implemented, tested in this order so that short dramatic events are considered before postural ones: **FALLING, JUMPING, CRAWLING, CROUCHING, SITTING, RUNNING, WALKING, STANDING**. Each event carries a track_id, a confidence, a start time and a duration.

3. Algorithms and computer vision techniques

Technique	Where	What it does
Ray-casting point-in-polygon	core/geometry.py	Tests whether a tracked object is inside a zone. Counts ray crossings; odd means inside. Works for concave polygons.
Segment intersection (orientation test)	core/line_crossing.py	Detects a virtual boundary crossing between two consecutive positions.
Zone debouncing	core/zone_engine.py	A zone change must persist before it is believed, so a box flickering on a boundary does not raise repeated intrusions.
Bottom-centre reference point	core/zone_engine.py	A person's position is their feet, a vehicle's is its box centre, so zone tests read where the object actually stands.
Normalised coordinates	throughout	All geometry is 0-1 rather than pixels, so a profile survives a resolution change.
Median filter (width 3)	core/movement.py	Removes single-frame pose spikes before speeds are measured. Added after a test showed one bad frame producing a JUMPING event from a stationary person.
Hysteresis (dual threshold)	core/low_light.py	Scene must fall below 60 to become low-light and rise above 80 to leave it. The gap stops the state flapping at dusk.
Rolling mean over 15 frames	core/low_light.py	Temporal smoothing of luminance.
INTER_AREA downscaling	core/low_light.py	Luminance measured on a 64px-wide copy. Area interpolation averages the pixels it collapses, so the small image is a fair summary.
80th-percentile reference height	core/pose_history.py	A person's standing height, robust to one bad ankle keypoint. Using the maximum would let a single bad frame corrupt the whole track.
Body-height normalisation	core/movement.py	Speed measured in the person's own body heights per second, making it approximately scale-free and camera-independent.
Temporal debouncing	core/behaviour_classifier.py	A candidate behaviour must hold 0.30 s (0.15 s for falls and jumps) before it becomes the person's state.
Additive weighted scoring	core/threat_engine.py	Threat score is a sum of configured factor weights, capped at 100. Pure function: same input, same output.
Incident deduplication and escalation	core/incident_manager.py	A repeat of the same event for the same track within 20 s raises the existing incident's score instead of opening a new one. This is what stops one person generating hundreds of alerts.
Morphological plate proposal	core/anpr.py	Blackhat, Sobel gradient, closing, Otsu threshold, contour filtering by aspect ratio to propose plate regions.
CLAHE + Otsu	core/anpr.py	Contrast-limited adaptive histogram equalisation and adaptive thresholding to prepare a plate crop for OCR.
WAL + single connection with RLock	database/database.py	Safe concurrent SQLite access from the worker thread.

4. Person activity analysis (the newest subsystem)

This is the four-stage chain that turns keypoints into named behaviour. Each stage is a separate module so each can be tested alone.

Stage	Module	Output
1. Pose estimation	core/pose.py	17 COCO keypoints per person, each with a confidence and a visibility flag, mapped back to full-frame normalised coordinates
2. Pose history	core/pose_history.py	3-second rolling buffer per Track ID, capped at 90 samples, plus that person's reference standing height
3. Movement analysis	core/movement.py	Speed, vertical rates, height ratio, hip ratio, torso angle, knee angle, direction, hand and limb geometry
4. Behaviour classification	core/behaviour_classifier.py	One debounced behaviour per Track ID, with confidence, duration and the evidence numbers behind it

The 17 tracked body landmarks

nose, left eye, right eye, left ear, right ear, left shoulder, right shoulder, left elbow, right elbow, left wrist, right wrist, left hip, right hip, left knee, right knee, left ankle, right ankle

Why speed is measured in body heights per second

A person ten metres from the camera is half the size of the same person at five metres, so a threshold in frame units means something different at every distance. Dividing by that person's own standing height makes the measurement roughly scale-free. The thresholds then come from human gait - a walk is about 1.4 m/s against a 1.7 m frame, a run comfortably over 3 m/s - rather than from tuning against one clip. Walking fires above 0.45, running above 1.9.

Cost

Component	Measured cost	Basis
Object detection	65-80 ms per analysed frame	YOLOv8n, CPU, 960x540 input
Pose estimation	12-16 ms per person crop	YOLOv8n-pose at 192px, batched, max 4 people per frame
Pose adds to the pipeline	+10% frame time	Same 200-frame clip with pose off and on
Activity chain	0.15-0.19 ms per person per analysed frame	Measured directly; too small to resolve in an end-to-end A/B
Low-light detection	0.12 ms per frame	Stride-slice then INTER_AREA, ~1.2% of one core for 4 cameras

5. Threat scoring

Threat is a separate layer from behaviour. An observation is not a verdict: a person running is a behavioural event, not automatically a critical threat. The engine sums configured factor weights and maps the total to a level, and every incident carries the list of reasons that produced its score.

Factor	Weight	Fires when
Restricted zone intrusion	30	Object inside a RESTRICTED polygon
Watchlist match	30	Plate matches a local watchlist entry
Border zone entry	25	Object inside a BORDER polygon
Virtual boundary crossing	20	Track crosses a configured line
Loitering	20	Dwell and movement-radius test both pass
Vehicle in no-vehicle zone	20	Vehicle inside a NO_VEHICLE polygon
Unauthorised vehicle	20	Vehicle in an alarming zone
Behavioural anomaly	15	Isolation Forest flags the track
Extended dwell	5 per minute, max 15	Loitering beyond threshold
Low-light conditions	10	Measured scene luminance, not the clock
Group presence	10	Three or more people tracked at once
Unusually fast movement	10	Speed above the configured threshold

Levels: LOW 0-24, MEDIUM 25-49, HIGH 50-74, CRITICAL 75-100.

Worth mentioning in the deck as a design decision. The low-light factor originally read the system clock against a configured night window. It now measures luminance from the frames themselves, because the clock is a poor proxy: a floodlit compound at 02:00 is not a low-light scene, an unlit bay at noon is, and recorded footage replayed the next morning is night video with a daytime timestamp.

6. Security geometry: optional and per-camera

Zones and lines are configured per camera in `config/zones.yaml` and selected by each camera's `zones_profile`. A camera that does not name a profile gets **no geometry at all**: it still runs detection, tracking, pose, behaviour, anomaly and threat scoring, but it cannot raise a zone intrusion, and no polygon is drawn. A person simply being visible is not a security event.

Zone kind	Meaning
RESTRICTED	Any object inside is an intrusion
BORDER	The line-of-control strip; entry is an intrusion
NO_VEHICLE	Only vehicles inside are a violation
PATROL	Friendly area; presence logged, never scored
OBSERVATION	General monitoring; presence logged, never scored

7. Technology stack

Layer	Technology	Version
Deep learning runtime	PyTorch	2.14
Detection and pose	Ultralytics YOLO	8.4
Classical ML	scikit-learn	1.8
Computer vision	OpenCV	4.11
Numerics	NumPy, SciPy, pandas	1.26 / 1.17 / 2.x
Model persistence	joblib	1.5
Desktop interface	PySide6 (Qt 6)	6.5+
Backend API	FastAPI + uvicorn	709-line server
Web dashboard	React + Vite	React 19, Vite 8
Database	SQLite (WAL mode)	13 tables, 18 indexes
Configuration	PyYAML	4 YAML files, no hard-coded parameters
Testing	pytest	269 tests

Two interfaces, one pipeline

The same `SurveillancePipeline` class drives both a PySide6 desktop application (9 screens: Dashboard, Cameras, Incidents, Alerts, Watchlist, Analytics, Evidence, Settings, System Logs) and a FastAPI backend serving a React dashboard over MJPEG. The pipeline module contains no Qt and no web framework code, which is what lets the same analysis run headless in the acceptance script.

REST endpoints

Method	Endpoint	Returns
GET	/api/health	Service status
GET	/api/cameras	Configured cameras and their state
POST	/api/cameras/{id}/start	Starts that camera's worker
POST	/api/cameras/{id}/stop	Stops it
GET	/api/cameras/{id}/stats	Live per-camera statistics
GET	/api/cameras/{id}/stream	MJPEG annotated video
GET	/api/dashboard	Aggregate counters
GET	/api/alerts	Alert feed
GET	/api/incidents	Incident records
GET	/api/plates	Plate reads

8. Validation and evidence

The project ships verification scripts rather than claims. Each one prints PASS or FAIL against a stated expectation.

Script	What it proves
run_acceptance.py	18 end-to-end checks: database, model load, class names read from the model, zones, tracking, zone and line events, loitering, incidents, alerts, evidence, analytics, incident lifecycle
check_fence_fixture.py	Scripted clip with a known answer: exactly 2 tracks and 3 incidents, all against the same person, and nothing at all against the patrol guard
check_zones.py	Geometry is optional and per-camera: identical footage produces zero zone events with no profile and real events with one
check_low_light.py	Low-light state follows the picture, not the clock: dimming the same clip flips it while a 12-hour clock change does not
check_pose.py	Pose adds keypoints and changes nothing else: identical Track IDs and identical incident counts with pose off and on
check_activity.py	Behaviour is temporal, not per-frame; separate histories per person; nothing fabricated where pose could not read
check_demo_cameras.py	All enabled cameras on real footage, with the architectural invariant checked

Test coverage by area

Test module	Tests
Database, config and inputs	50
Anomaly, threat and incidents	47
Geometry, zones and lines	45
Pose estimation	29
Person activity	26
Tracking and behaviour	25
End-to-end acceptance chain	18
Low-light detection	18
Zone optionality	11
Total	**269**

9. What is NOT implemented

This section exists so nothing here reaches a slide by accident. Presenting any of these as working would be the fastest way to lose credibility in a judging session.

Capability	Actual state
Weapon detection	**NOT IMPLEMENTED.** No weapon model is installed. The COCO classes in YOLOv8n do not include firearms. A dedicated weapon dataset and model would be required.
Face detection / recognition	**NOT IMPLEMENTED.** No face model exists in the project.
ANPR optical character recognition	**PARTIAL.** The full plate-detection and image-preprocessing pipeline is built and correctly gated to vehicle tracks, but no OCR backend (EasyOCR or Tesseract) is installed, so plates are recorded as UNREADABLE rather than guessed.
Trained action-recognition model	**NOT IMPLEMENTED.** Behaviour classification is rule-based.
Phase 2 behaviours	**NOT IMPLEMENTED.** Sudden movement, repeated approach and retreat, and abnormal movement are designed but not built.
Speed in metres per second	Requires a per-camera pixels-per-metre calibration that has not been measured. Speed is reported in frame-relative units, which is honest but not physically calibrated.
Automatic zone suggestion from imagery	**NOT IMPLEMENTED** and deliberately so. Pixels cannot tell you that a region is legally restricted.

Known limitations of what IS implemented

Limitation	Detail
Walking along the camera axis	Speed is image-plane speed, so a person walking directly towards or away from the camera barely moves in frame and reads as STANDING. Measured: an officer plainly walking moved 0.30 of the frame in 16 seconds at 0.46 frame height, about 0.04 body heights per second. Lowering the threshold would misclassify stationary people.
Gait detection as the fix	Investigated and measured, not shipped. Ankle-separation standard deviation was 0.013-0.023 of body height for stationary people and 0.045-0.068 for moving ones, but the highest reading of all, 0.149, belonged to a track that never moved - a partly occluded figure with badly estimated ankles. Needs a confidence gate and a periodicity test first.
Sitting versus crouching	Both lower the hips and bend the knees; the difference is largely what the person is resting on, which a skeleton does not show.
Lying down versus approaching the lens	Both foreshorten the torso identically on one uncalibrated camera.
Mean luminance on high-contrast scenes	A sunset scene with the sun in frame read 145 - brighter than daylight clips - because the sky dominates the mean, even though the ground where people walk is dark.
Anomaly model is global, not per camera	The Isolation Forest is fitted across cameras. A model trained on one scene flags ordinary behaviour in a different scene. Arguably the same class of problem as global zones.
Demo footage licensing	The current demo clips are Getty Images preview files with watermarks burned in. They are usable for development only and MUST be replaced with own-filmed or properly licensed footage before submission.

10. Suggested slide points

Strongest technical claims, all backed by evidence in this document:

- One camera-agnostic AI pipeline; zero camera-specific branching in 20,000 lines of code.
- Four-stage person activity chain - pose, history, movement, behaviour - with eight classified actions and temporal debouncing.
- Scale-free measurement: speeds in body heights per second, so thresholds work at any camera distance without calibration.
- Explainable threat scoring: every incident carries the list of weighted factors that produced its score.
- Incident deduplication: one event produces one incident, not four hundred alerts.
- 269 automated tests, plus seven verification scripts that print pass or fail against stated expectations.
- Low-light context measured from pixels rather than the system clock - a correctness fix with a demonstrable before and after.
- Proven negative controls: on real footage of a patrol officer doing his job the system raises nothing, which is as important as detecting the intruder.

Phrases to avoid

- “Detects weapons” - no weapon model exists.
- “Facial recognition” - not implemented.
- “Reads number plates” - the pipeline exists but no OCR backend is installed; say “plate localisation implemented, OCR pending”.
- “Deep-learning behaviour recognition” - the behaviour layer is rule-based; the deep learning is in detection and pose.
- “Trained on border-force data” - the models are stock COCO weights and the footage is public stock video.

One honest framing that plays well. The system is built so that it can be argued with. Every threshold is in a YAML file, every alert carries its reasons, every claim has a script that checks it, and the limitations are written into the code rather than discovered by a judge. That is a stronger position than a demo that appears to do everything.